

TravelNest Performance Optimization Report

Yeh report un tamaam Performance Optimization rules ka mukammal analysis hai jo humne check kiye hain. Is mein bataya gaya hai ke konsi guide codebase mein 100% Implemented hai, konsi Partially Implemented hai, aur konsi Not Implemented hai.

Part A - Fixing Local Development Speed

- 3.1 Switch to Turbopack (Next.js 14 Dev Mode): [Not Implemented] - 'dev' script 'next dev -p 3000' hai, --turbo flag nahi hai.
- 3.2 Reduce node_modules Overhead: [Not Implemented] - pnpm ke bajaye standard npm use ho raha hai.
- 3.3 Fix Slow Hot Reload (Fast Refresh): [Partially Implemented] - Heavy libraries root layout mein nahi hain, lekin barey Client Components ko split nahi kiya gaya.
- 3.4 Local Supabase/Database Latency: [Not Implemented] - Local dev server abhi bhi remote (hosted) Supabase se connect ho raha hai.
- 3.5 TypeScript & ESLint Overhead: [Partially Implemented] - 'type-check' script mojud hai, lekin Next.js ki default type-checking ko disable nahi kiya gaya.

Part B - Fixing Vercel Production Speed & Rendering

- 4.1 Choose the Right Rendering Strategy Per Page: [100% Implemented] - Homepage/ Destinations par ISR (revalidate=3600) hai. Search par SSR (force-dynamic) hai.
- 4.2 On-Demand Revalidation: [100% Implemented] - /api/revalidate/route.ts route bilkul perfectly secret-key verified on-demand revalidation use kar raha hai.
- 4.3 Reduce JavaScript Bundle Size: [Partially Implemented] - next/dynamic ka use TourGallery ke liye hua hai, lekin Maps ke liye native import use hua hai.

Part D - Database & Caching Optimizations

- 4.5 Cache-Control Headers for API Routes: [Partially Implemented] - 'listings' API mein Cache header hai, lekin 'destinations' API mein success response se miss ho gaya hai.
- 5.1 Connection Pooling (Critical for Serverless): [Not Applicable] - Supabase ka REST API client use ho raha hai jo connection pool khud handle karta hai.
- 5.2 Add Indexes on Frequently Queried Columns: [Not Implemented] - supabase-schema.sql mein required performance-based indexes shamil nahi hain.
- 5.3 Select Only What You Need: [Not Implemented] - Taqreeban 30 se zyada jaghon par 'supabase.from(...).select(*)' use ho raha hai.
- 5.4 Row-Level Security (RLS) Performance: [Not Implemented] - setup-rls.sql mein slow syntax USING (auth.uid)::text use ho raha hai, optimize nahi kiya gaya.
- 5.5 Avoid N+1 Query Patterns: [Not Implemented] - Nested joins ki bajaye 'waterfall' pattern use hua hai (Tours ke baad alag se reviews fetch hotay hain).

Part E - Realtime & UI Speed

- 6.1 Scope Channels Narrowly: [100% Implemented] - Real-time database channels par strictly filter lagaye gaye hain (e.g., filter: 'supplier_id=eq...').
- 6.2 Always Clean Up Subscriptions: [100% Implemented] - Har channel ko useEffect ke cleanup mein removeChannel() ke zariye close kiya gaya hai.
- 6.4 Optimistic UI Updates: [Not Implemented] - Actions (jaise Add to Wishlist ya Cancel Booking) UI ko instant update nahi karte, server ka intezaar hota hai.

Part F - NestJS Backend API Speed

- 7.1 Add Response Compression: [100% Implemented] - NestJS main.ts mein app.use(compression()) successfully applied hai.
- 7.2 Add a Caching Layer (Redis): [100% Implemented] - app.module.ts mein CacheModule configured hai aur controllers mein @CacheTTL applied hai.
- 7.3 Make Independent Operations Parallel: [100% Implemented] - Backend mein Promise.all use kar ke independent APIs parallel laye ja rahay hain.
- 7.4 Validate Fast, Fail Fast: [100% Implemented] - class-validator DTOs aur ValidationPipe configured hain, aur rate-limiting bhi implemented hai.

Part G - Image, Font & Asset Optimization

- 8.1 Images via Cloudinary + next/image: [Partially Implemented] - next.config.mjs mein Cloudinary allowed hai, lekin 42 jaghon par abhi bhi old tags use ho rahay hain.
- 8.2 Fonts: [100% Implemented] - Google fonts ko next/font/google ke zariye theek se self-host kiya gaya hai.
- 8.3 Third-Party Scripts: [100% Implemented] - External widgets ke liye next/script strategy='afterInteractive' ke sath use hua hai.